

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT on

Machine Learning (23CS6PCMAL)

Submitted by

RAJINDER KUMAR (1BM23C260)

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Jan-2026 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **RAJINDER KUMAR (1BM23CS260)**, who is a bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Rashmi H Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	2-2-2026	Write a python program to import and export data using Pandas library functions	2
2	9-2-2026	Demonstrate various data pre-processing techniques for a given dataset	4
3	16-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	6
4	16-2-2026	Build Logistic Regression Model for a given dataset	8
5	2-3-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	11
6	23-2-2026	Build KNN Classification model for a given dataset.	13
7	9-3-2026	Build Support vector machine model for a given dataset	16
8	16-3-2026	Implement Random forest ensemble method on a given dataset.	19
9	23-3-2026	Implement Boosting ensemble method on a given dataset.	21
10	20-4-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	23
11	20-4-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	25

Github Link:

<https://github.com/rajinderkumar143/ML-lab.git>

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

Week - 1

(a) Task 1: Load the dataset
 from sklearn.datasets, import load_iris
 import pandas as pd
 iris = load_iris()
 df = pd.DataFrame(iris.data, columns=iris.feature_names)
 df['species'] = iris.target
 df.head()

(b) Task 2: Understand the shape

df.shape
 (150, 5)

Observation Questions

(i) How many rows (examples)?

Ans: 150

(ii) How many features?

Ans: 5

(c) Task 3: Explore Columns

df.columns

df.info()

<class 'pandas.core.DataFrame'>

Range Index: 150 entries, 0 to 149

Data Columns (total 5 columns):

#	column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	species	150 non-null	int64

d types: float64 (4),
 memory usage:

int64 (1)
 6.0 KB.

Task 4: Descriptive Summary

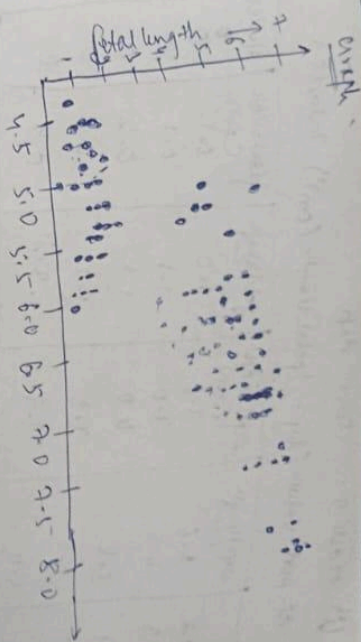
	df	mean	std	var	skewness	kurtosis
sepal length (cm)	149	5.843	0.8280	0.6856	-0.1173	0.0000
petal length (cm)	149	4.2652	1.0601	1.1239	0.0000	0.0000
petal width (cm)	149	0.9422	0.0892	0.0079	0.0000	0.0000
species	1	0.0000	0.0000	0.0000	0.0000	0.0000

Task 5: Count each category of 'species'.

Species	Count
0	50
1	50
2	50

Task 6: Plot one variable

import matplotlib.pyplot as plt
 plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
 plt.xlabel('sepal length (cm)')
 plt.ylabel('petal length (cm)')
 plt.show()



Task 7: Compare two species
 df[df['species'] == 0].head()

df[df['species'] == 1].head()

	sepal length (cm)	petal length (cm)	species
50	4.9	1.4	0
51	5.0	1.5	0
52	5.1	1.6	0
53	5.2	1.7	0
54	5.3	1.8	0

Task 8: Correlation

	sepal length (cm)	petal length (cm)	species
sepal length (cm)	1.000	0.8719	0.9825
petal length (cm)	-0.1173	1.000	0.9985
species	0.8719	0.9985	1.000

(i) Task 9: Sort data.

df.sort_values(by = 'petal length (cm)', inplace=True)

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

8/12/26

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
4.6	3.6	1.0	0.2	0
4.3	3.0	1.1	0.1	0
5.8	4.0	1.2	0.2	0
5.0	3.2	1.2	0.2	0
5.4	3.9	1.3	0.4	0

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
4.6	3.6	1.0	0.2	0
4.3	3.0	1.1	0.1	0
5.8	4.0	1.2	0.2	0
5.0	3.2	1.2	0.2	0
5.4	3.9	1.3	0.4	0

Code:

```
import pandas as pd
import numpy as np
```

```
adult = pd.read_csv("sample_data/adult.csv")
adult.info()
# Replace ? with NaN
adult.replace("?", np.nan, inplace=True)
```

```
for col in adult.columns:
```



```

if adult[col].dtype == 'object':
    adult[col].fillna(adult[col].mode()[0],
                     inplace=True)
else:
    adult[col].fillna(adult[col].mean(), inplace=True)

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for col in adult.select_dtypes(include='object').columns:
    adult[col] = le.fit_transform(adult[col])
num_cols = adult.select_dtypes(include=['int64', 'float64']).columns

Q1 = adult[num_cols].quantile(0.25)
Q3 = adult[num_cols].quantile(0.75)
IQR = Q3 - Q1
adult = adult[~(
    (adult[num_cols] < (Q1 - 1.5 * IQR)) |
    (adult[num_cols] > (Q3 + 1.5 * IQR))
).any(axis=1)]

from sklearn.preprocessing import MinMaxScaler
minmax_scaler = MinMaxScaler()
adult_minmax = adult.copy()
adult_minmax[num_cols] = minmax_scaler.fit_transform(adult[num_cols])

from sklearn.preprocessing import StandardScaler
std_scaler = StandardScaler()
adult_standard = adult.copy()
adult_standard[num_cols] = std_scaler.fit_transform(adult[num_cols])

```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot

Week - 2

1. End-to-End Machine Learning

- (i) Working with real data
- (ii) look at the big picture
- (iii) cut the data, discover
- (iv) and visualize the data.
- (v) Prepare the data, select.
- (vi) and train the model
- (vii) Fine tune your model

Application :- House Price Prediction.

Step 0 :- Import required libraries

Step 1 :- Look at the Big Picture

Here, we defined ML problem clearly:

(i) Supervised Regression.

(ii) Target: MedianHouseVal

Observations :-

- (i) Correct choice of metrics for Regression.
- (ii) RMSE penalizes large errors $\rightarrow$ good for house prices.
- (iii) R^2 help explain variance, but shouldn't be used alone.

Step 2 :- Cut the Data

Step 3 :- Discover the Data (EDA)

Step 4 :- Visualize the Data

4.1 Distribution of House Prices

4.2 Income vs House Price

4.3 Correlation heatmap

4.4 Location-based Visualization.

Step 5 - Prepare the data

5.1 Feature Engineering

5.2 Train-Test Split

5.3 Feature Scaling

Step 6 - Baseline Model - Linear Regression

Step 7 - Advanced Models

7.1 Decision Tree

7.2 Random Forest

Step 8 - Fine-Tuning (Grid Search)

Step 9 - Model Comparison

~~Linear Regression~~

Q1 - Which model performed best?

Ans: The tuned Random Forest model performed best, as it achieved the lowest RMSE on the test set, indicating the smallest prediction error compared to linear regression, decision tree, and the untuned Random forest.

Q2 - Why did Random Forest outperform linear regression?

Ans: Random Forest can capture non-linear relationships and feature interactions, while linear regression assumes a linear relationship, which is too simple for housing price data.

Q3 - Why did hyperparameter tuning improve performance?

Ans: Tuning helped control overfitting and underfitting by optimizing parameters like tree depth and nb. of trees, leading to better generalization on test data.

Q4 - What additional data could improve the model?

Ans: Location-based features, crime rates, school quality, economic indicators, and market trends could further improve predictions.

Q5 -

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
housing=
pd.read_csv("https://media.geeksforgeeks.org/wp-content/uploads/20240319120216/housing.csv")
housing.info()
housing.describe()
housing.hist(bins=50, figsize=(20,15))
plt.show()
```

```
import matplotlib.pyplot as plt
```

```
housing.plot(kind="scatter",x="longitude",y="latitude",alpha=0.4,s=housing["population"] /
100,label="Population",figsize=(10,7),c="median_house_value",cmap="jet",colorbar=True)
```

```
plt.legend()
```



```
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Geographical Distribution of Housing Prices in California")
plt.show()
```

```
corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
housing.plot(kind="scatter", x="median_income", y="median_house_value", alpha=0.1, figsize=(8,6))
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Correlation between Median Income and Housing Price")
plt.show()
```

```
housing["rooms_per_household"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] / housing["total_rooms"]
housing["population_per_household"] = housing["population"] / housing["households"]
```

```
corr_matrix = housing.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
housing.plot(kind="scatter", x="rooms_per_household", y="median_house_value",
alpha=0.1, figsize=(8,6))
```

```
plt.xlabel("Rooms per Household")
plt.ylabel("Median House Value")
plt.title("Improved Correlation using Feature Combination")
plt.show()
```

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
```

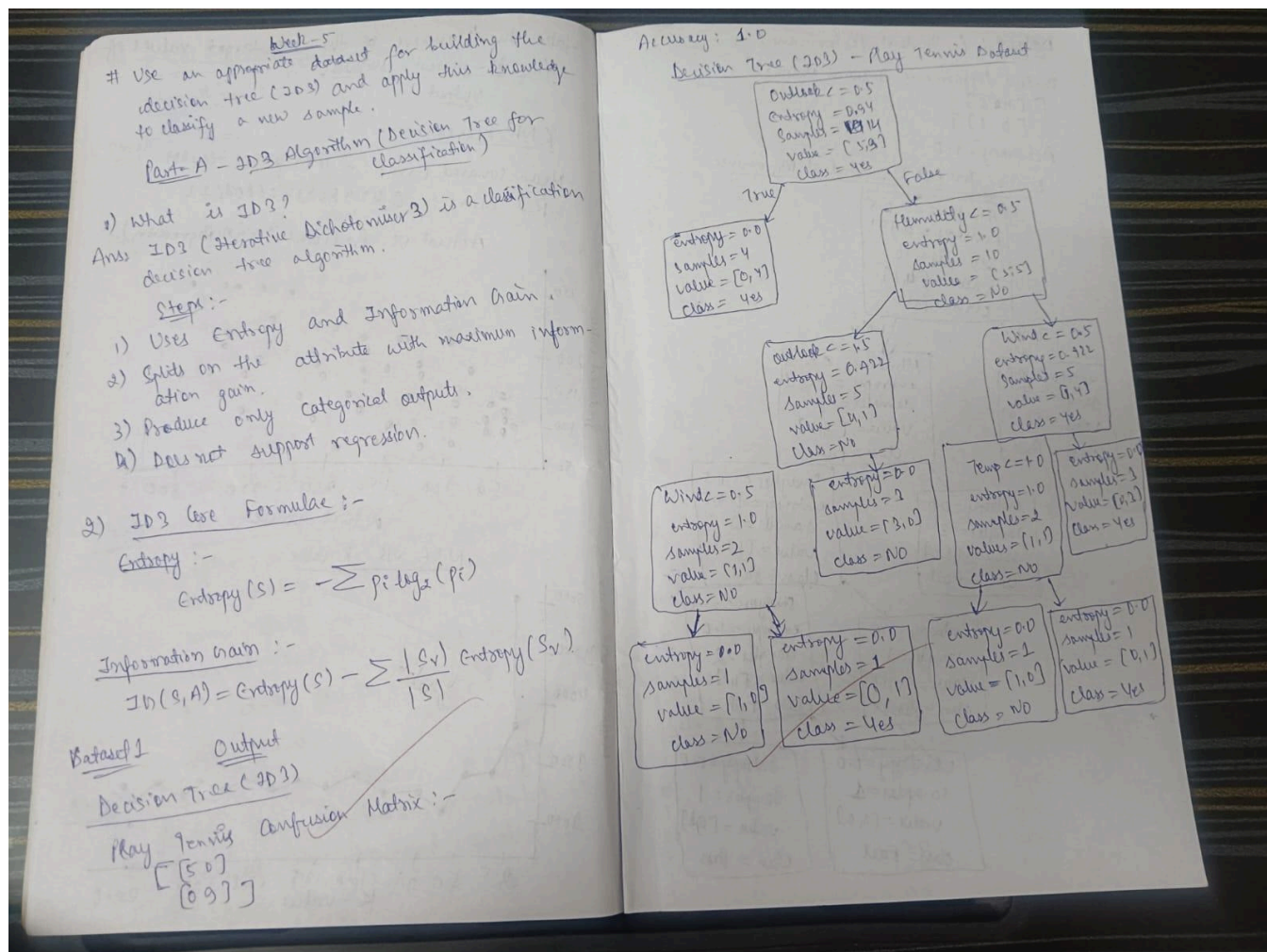
```
housing['total_bedrooms'].fillna(housing['total_bedrooms'].median(), inplace=True)
housing = pd.get_dummies(housing, columns=['ocean_proximity'], drop_first=True)
num_cols = [
    'longitude', 'latitude', 'housing_median_age', 'total_rooms',
    'total_bedrooms', 'population', 'households', 'median_income'
]
```

```
scaler = StandardScaler()
housing[num_cols] = scaler.fit_transform(housing[num_cols])
housing.columns = housing.columns.str.strip()
housing_encoded = pd.get_dummies(housing, columns=['ocean_proximity'], drop_first=True)
print(housing_encoded.head())
```

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot



Code:

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import mean_absolute_error, mean_squared_error
import math
```

```
df1 = pd.read_csv("drug.csv")
df2 = pd.read_csv("iris.csv")
```

```
df3 = pd.read_csv("petrol_consumption.csv")
```

```

X_iris = df2.drop("species", axis=1)
y_iris = df2["species"]
X_train, X_test, y_train, y_test = train_test_split(X_iris, y_iris, test_size=0.2, random_state=42)
iris_model = DecisionTreeClassifier()
iris_model.fit(X_train, y_train)
y_pred = iris_model.predict(X_test)
print("IRIS Accuracy:", accuracy_score(y_test, y_pred))
print("IRIS Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

```

le = LabelEncoder()
df1["Sex"] = le.fit_transform(df1["Sex"])
df1["BP"] = le.fit_transform(df1["BP"])
df1["Cholesterol"] = le.fit_transform(df1["Cholesterol"])
df1["Drug"] = le.fit_transform(df1["Drug"])

```

```

X_drug = df1.drop("Drug", axis=1)
y_drug = df1["Drug"]
X_train, X_test, y_train, y_test = train_test_split(X_drug, y_drug, test_size=0.2, random_state=42)
drug_model = DecisionTreeClassifier()
drug_model.fit(X_train, y_train)

```

```

y_pred = drug_model.predict(X_test)
print("\nDrug Dataset Accuracy:", accuracy_score(y_test, y_pred))
print("Drug Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

```

```

X = df3.drop("Petrol_Consumption", axis=1)
y = df3["Petrol_Consumption"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
reg_model = DecisionTreeRegressor()
reg_model.fit(X_train, y_train)
y_pred = reg_model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = math.sqrt(mse)

```

```

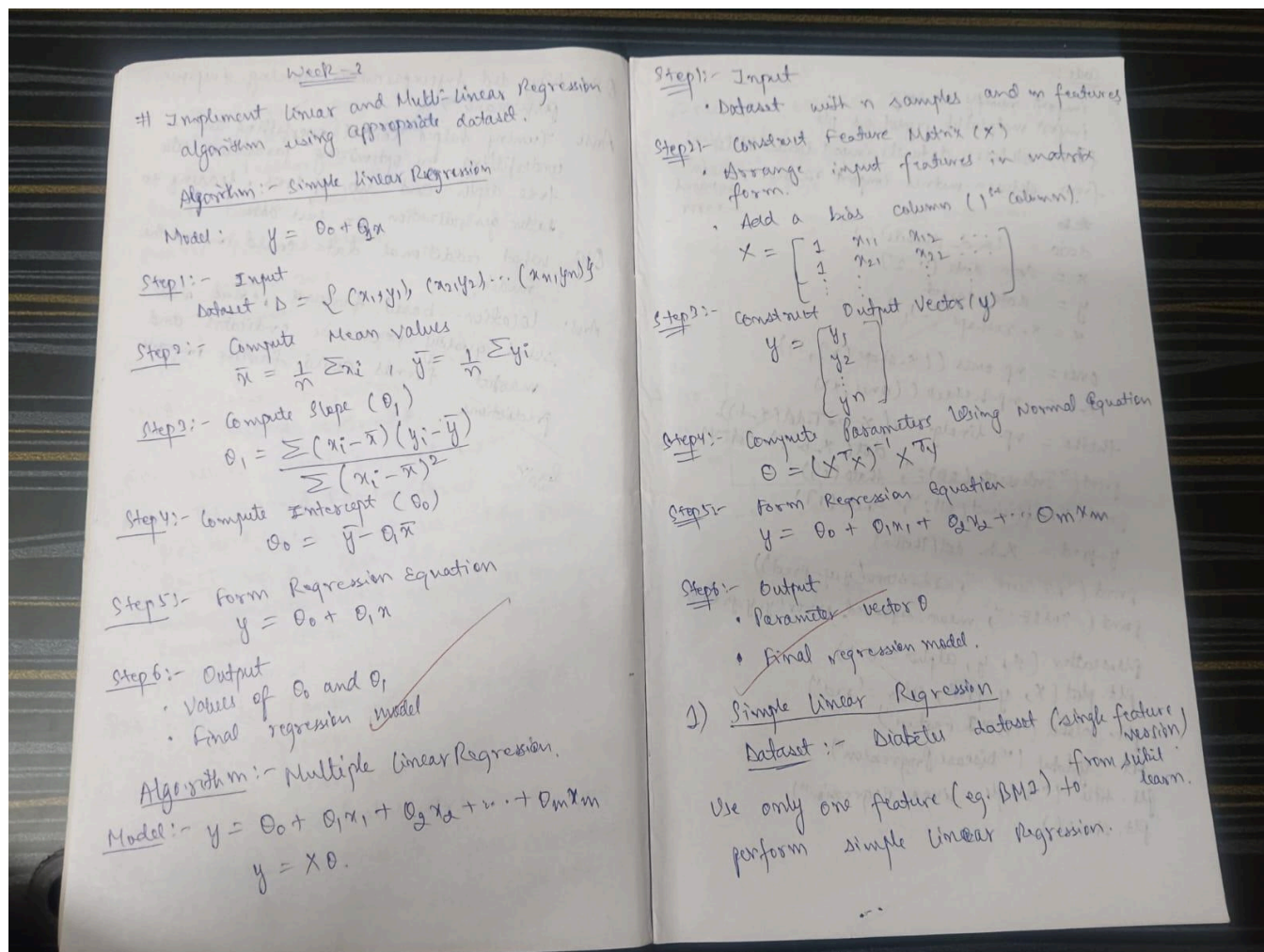
print("Mean Absolute Error:", mae)
print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)

```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot



Code:-

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.metrics import r2_score, mean_squared_error

data = load_diabetes()
X = data.data[:, 2]
y = data.target
X = X.reshape(-1, 1)

ones = np.ones((X.shape[0], 1))
X_b = np.hstack((ones, X))
theta = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T.dot(y))

print("Intercept (00):", theta[0])
print("Coefficient (01):", theta[1])

y_pred = X_b.dot(theta)

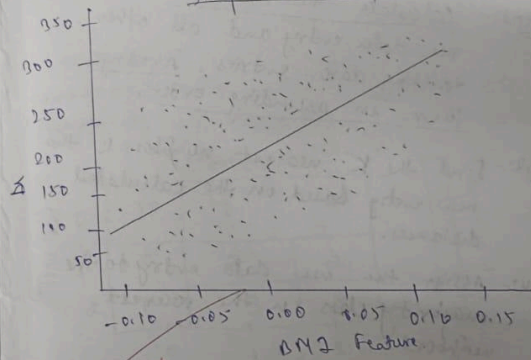
print("R2 score:", r2_score(y, y_pred))
print("MSE:", mean_squared_error(y, y_pred))

plt.scatter(X, y, alpha=0.5)
plt.plot(X, y_pred, color='red')
plt.xlabel("BMI Feature")
plt.ylabel("Disease Progression")
plt.title("Simple Linear Regression")
plt.show()
```

Output

Intercept (00): 152.13338
Coefficient (01): 349.8935
R2 score: 0.34392
MSE: 3890.4565

Simple Linear Regression



~~Output~~ ~~Simple Linear Regression~~

Code:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

df = pd.read_csv('/content/sample_data/canada_per_capita_income.csv')
df.info();
X = df[['year']]
y = df['per capita income (US$)']

model = LinearRegression()
model.fit(X, y)
```

```

m = model.coef_[0]
b = model.intercept_

print("Linear Regression Equation:")
print(f'y = {m:.4f} * x + {b:.4f}')
predicted_income = model.predict([[2020]])
print("Predicted per capita income in 2020:", predicted_income[0])

```

```

plt.scatter(X, y, color='red')
plt.plot(X, model.predict(X), color='blue')
plt.xlabel("Year")
plt.ylabel("Per Capita Income (US$)")
plt.title("Year vs Per Capita Income")
plt.show()

```

```

X = df['year'].values
Y = df['per capita income (US$)'].values
N = len(X)
x_mean = np.mean(X)
y_mean = np.mean(Y)
xy_mean = np.mean(X * Y)
x2_mean = np.mean(X ** 2)

```

```

m = (xy_mean - x_mean * y_mean) / (x2_mean - x_mean ** 2)
b = y_mean - m * x_mean

```

```

print("Linear Regression Equation:")
print(f'y = {m:.4f} * x + {b:.4f}')
predicted_income = m * 2020 + b
print("Predicted per capita income in 2020:", predicted_income)

```

```

df2 = pd.read_csv('/content/sample_data/salary.csv')
df2.info()
print(df2.isnull().sum())
df2['YearsExperience'].fillna(df2['YearsExperience'].mean(), inplace=True)
X = df2[['YearsExperience']]
y = df2['Salary']

```

```

model = LinearRegression()
model.fit(X, y)
m = model.coef_[0]
b = model.intercept_
print("Linear Regression Equation:")
print(f'Salary = {m:.4f} * YearsExperience + {b:.4f}')

```

```
predicted_salary = model.predict([[12]])
print("Predicted Salary for 12 years experience:", predicted_salary[0])
df2['YearsExperience'].fillna(df2['YearsExperience'].mean(), inplace=True)
```

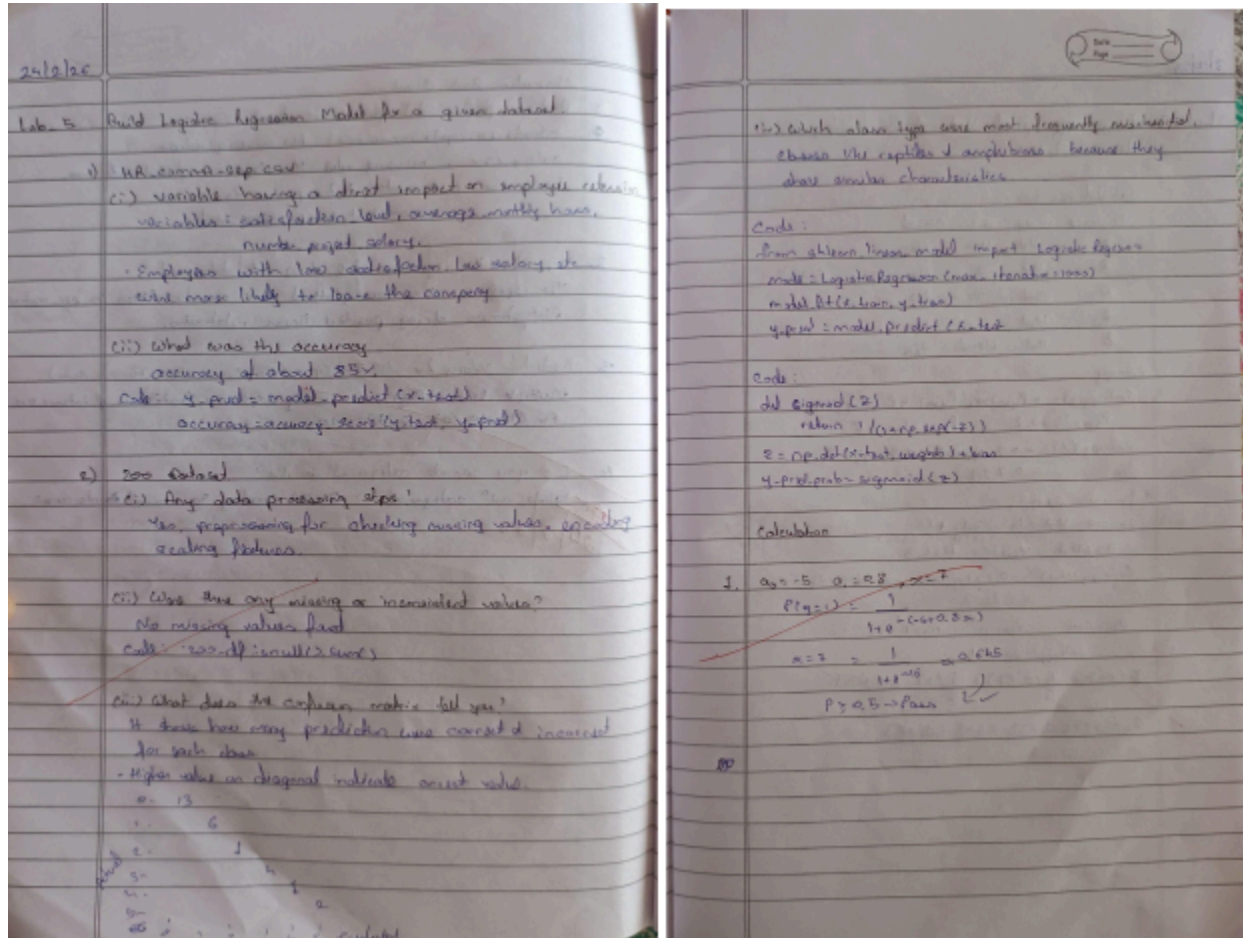
```
X = df2['YearsExperience'].values
Y = df2['Salary'].values
N = len(X)
x_mean = np.mean(X)
y_mean = np.mean(Y)
```

```
xy_mean = np.mean(X * Y)
x2_mean = np.mean(X ** 2)
m = (xy_mean - x_mean * y_mean) / (x2_mean - x_mean ** 2)
b = y_mean - m * x_mean
print("Linear Regression Equation:")
print(f"Salary = {m:.4f} * YearsExperience + {b:.4f}")
predicted_salary = m * 12 + b
print("Predicted Salary for 12 years experience:", predicted_salary)
```

Program 5

Build Logistic Regression Model for a given dataset

Screenshot



Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
df = pd.read_csv("/content/sample_data/HR_comma_sep.csv")
print(df['left'].value_counts()) # Target variable: 1 = left, 0 = stayed
```

```
# Map salary to numeric
df['salary'] = df['salary'].map({'low':0, 'medium':1, 'high':2})
```

```

df.columns = df.columns.str.strip()

if 'Department' in df.columns:
    df = pd.get_dummies(df, columns=['Department'], drop_first=True)
else:
    print("'Department' column not found! Columns available:", df.columns.tolist())

numeric_df = df.select_dtypes(include=['int64', 'float64'])
corr_matrix = numeric_df.corr()
print("\nCorrelation with 'left':\n", corr_matrix['left'].sort_values(ascending=False))

sns.countplot(x='salary', hue='left', data=df, palette='Set2')
plt.title("Impact of Salary on Employee Retention")
plt.xlabel("Salary Level (0=low,1=medium,2=high)")
plt.ylabel("Count of Employees")
plt.show()

sns.countplot(x='Department', hue='left',
data=pd.read_csv("/content/sample_data/HR_comma_sep.csv"), palette='Set1')
plt.title("Employee Retention by Department")
plt.xlabel("Department")
plt.ylabel("Count of Employees")
plt.xticks(rotation=45)
plt.show()

features = ['satisfaction_level', 'last_evaluation', 'number_project',
            'average_monthly_hours', 'time_spend_company', 'salary', 'promotion_last_5years']

dept_columns = [col for col in df.columns if col.startswith('Department_')]
features += dept_columns

X = df[features]
y = df['left']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy of Logistic Regression model: {accuracy:.4f}")

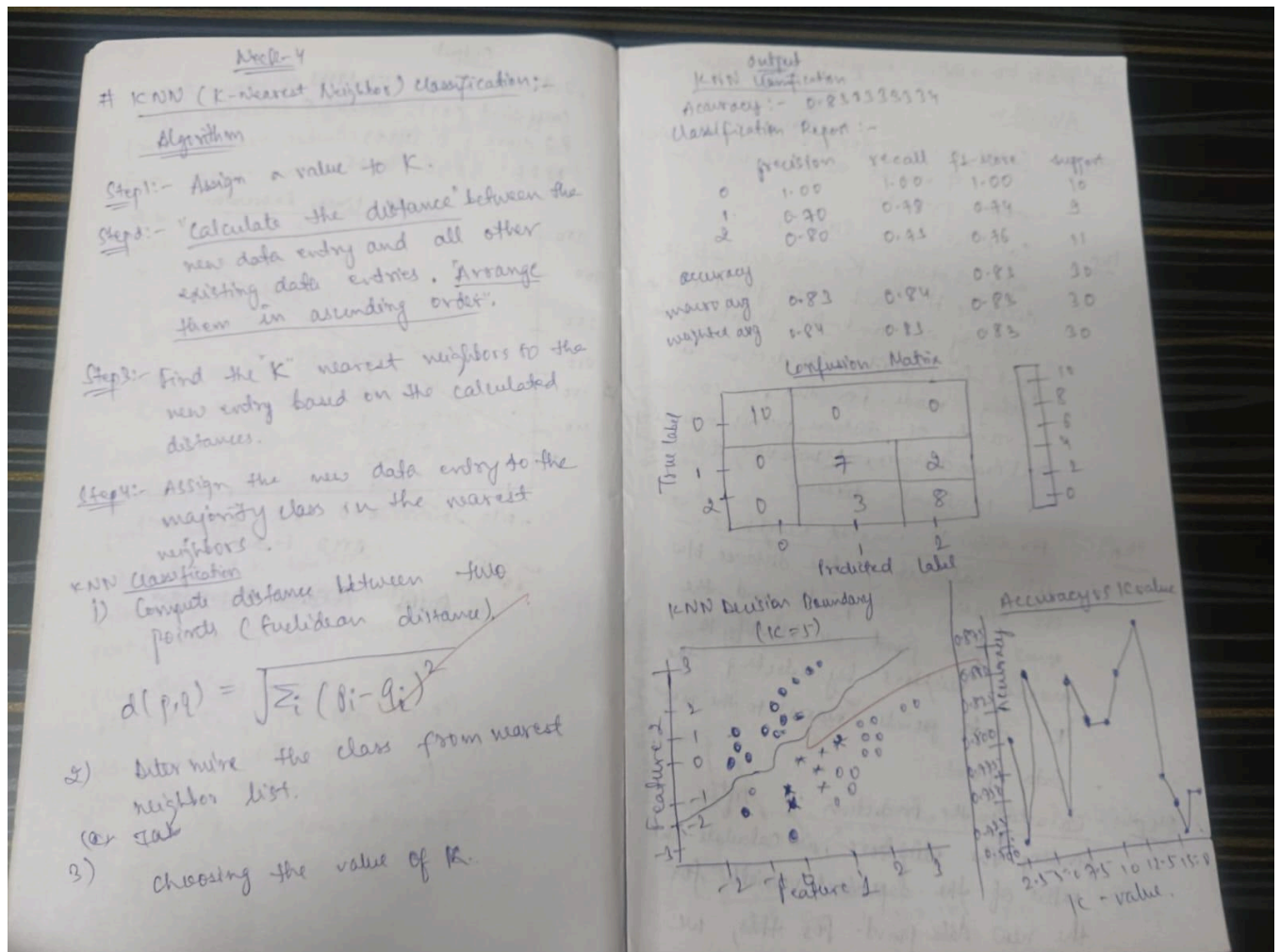
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

Program 6

Build KNN Classification model for a given dataset.

Screenshot



#1 KNN (K-Nearest Neighbor) Regression:-

Algorithm

Step 1:- Choose a value for K :- this determines the no. of nearest neighbors used to make prediction.

Step 2:- Calculate the distance:- After choosing K , we calculate the distance b/w each data point in the training set and the targeted data point for which a prediction is being made. For this, we can use a variety of distance metrics, including Euclidean distance, Manhattan distance or Minkowski distance.

Step 3:- Find the K -nearest neighbors:- After calculating the distance b/w the existing data points and the new data point, we identify K nearest neighbors by selecting the " K " data points nearest to the new data point.

Step 4:- Calculate the Prediction:- After finding the neighbors, we calculate the value of the dependent variable for the new data point. For this, we

take the "average of the target values of the K -nearest neighbors".

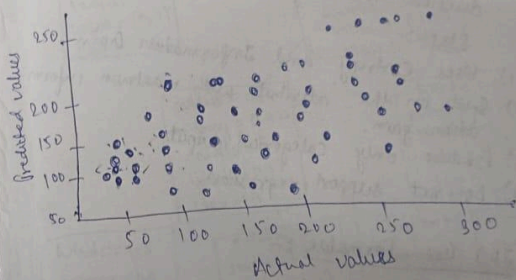
Output

KNN Regression

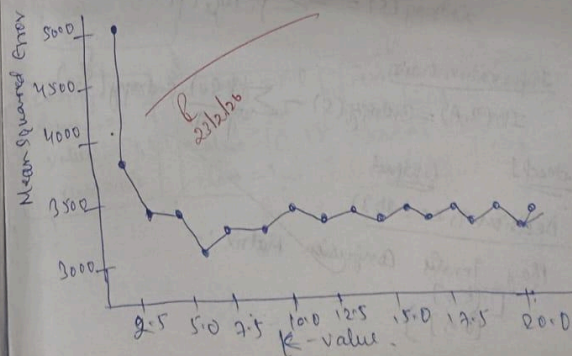
Mean Squared Error :- 3049.449587661993

R^2 score :- 0.4248088706606621

Actual vs Predicted (KNN Regression)



MSF v/s K value



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
ConfusionMatrixDisplay

df1 = pd.read_csv("diabetes.csv")
```

```

df2 = pd.read_csv("iris.csv")
df3 = pd.read_csv("heart.csv")

X_iris = df2.drop("species", axis=1)
y_iris = df2["species"]
X_train, X_test, y_train, y_test = train_test_split(X_iris, y_iris, test_size=0.2, random_state=42)

knn_iris = KNeighborsClassifier(n_neighbors=5) # k=5 is typical for IRIS
knn_iris.fit(X_train, y_train)
y_pred = knn_iris.predict(X_test)
print("IRIS Dataset Accuracy:", accuracy_score(y_test, y_pred))
print("IRIS Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("IRIS Classification Report:\n", classification_report(y_test, y_pred))

X_diabetes = df1.drop("Outcome", axis=1)
y_diabetes = df1["Outcome"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_diabetes)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_diabetes, test_size=0.2,
random_state=42)

knn_diabetes = KNeighborsClassifier(n_neighbors=7) # k=7 chosen after testing
knn_diabetes.fit(X_train, y_train)
y_pred = knn_diabetes.predict(X_test)
print("\nDiabetes Dataset Accuracy:", accuracy_score(y_test, y_pred))
print("Diabetes Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Diabetes Classification Report:\n", classification_report(y_test, y_pred))

X_heart = df3.drop("target", axis=1)
y_heart = df3["target"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_heart)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_heart, test_size=0.2, random_state=42)

# Find best k (1-20) based on accuracy
scores = []
k_values = range(1, 21)
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    scores.append(knn.score(X_test, y_test))

best_k = k_values[np.argmax(scores)]
print("Best k:", best_k, "with accuracy:", max(scores))

knn_heart = KNeighborsClassifier(n_neighbors=best_k)
knn_heart.fit(X_train, y_train)

```

```
y_pred = knn_heart.predict(X_test)

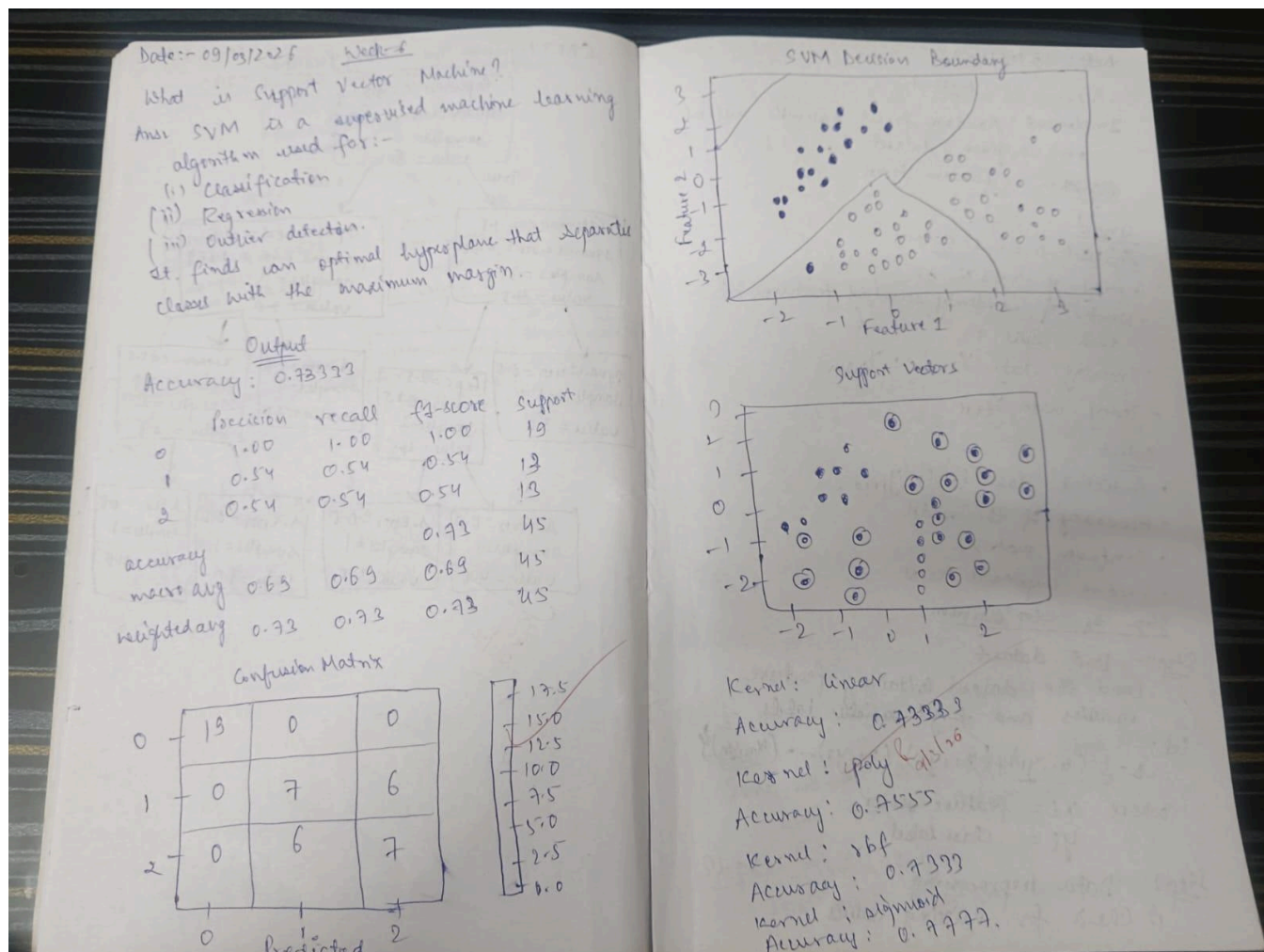
print("\nHeart Dataset Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

ConfusionMatrixDisplay.from_estimator(knn_heart, X_test, y_test, cmap='Blues')
plt.title("Heart Dataset Confusion Matrix")
plt.show()
```

Program7

Build Support vector machine model for a given dataset

Screenshot



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc
from sklearn.preprocessing import label_binarize

dfl = pd.read_excel("iris.xlsx")
```



```

df2 = pd.read_excel("letter-recognition.xlsx")
print(df1.info())
print(df2.info())

X_iris = df1.drop("species", axis=1)
y_iris = df1["species"]
X_train, X_test, y_train, y_test = train_test_split(X_iris, y_iris, test_size=0.2, random_state=42)

svm_linear = SVC(kernel='linear')
svm_linear.fit(X_train, y_train)
y_pred = svm_linear.predict(X_test)
print("IRIS Linear Kernel Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

ConfusionMatrixDisplay.from_estimator(svm_linear, X_test, y_test)
plt.title("IRIS Confusion Matrix - Linear Kernel")
plt.show()

# RBF Kernel
svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(X_train, y_train)
y_pred = svm_rbf.predict(X_test)
print("\nIRIS RBF Kernel Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

ConfusionMatrixDisplay.from_estimator(svm_rbf, X_test, y_test)
plt.title("IRIS Confusion Matrix - RBF Kernel")
plt.show()

X_letter = df2.drop("letter", axis=1)
y_letter = df2["letter"]
X_train, X_test, y_train, y_test = train_test_split(X_letter, y_letter, test_size=0.2, random_state=42)
svm_letter = SVC(kernel='rbf', probability=True)
svm_letter.fit(X_train, y_train)
y_pred = svm_letter.predict(X_test)
print("Letter Dataset Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

ConfusionMatrixDisplay.from_estimator(svm_letter, X_test, y_test)
plt.title("Letter Dataset Confusion Matrix")
plt.show()

y_test_bin = label_binarize(y_test, classes=np.unique(y_letter))
y_score = svm_letter.predict_proba(X_test)

# ROC for first class
fpr, tpr, _ = roc_curve(y_test_bin[:,0], y_score[:,0])

```

```
roc_auc = auc(fpr, tpr)

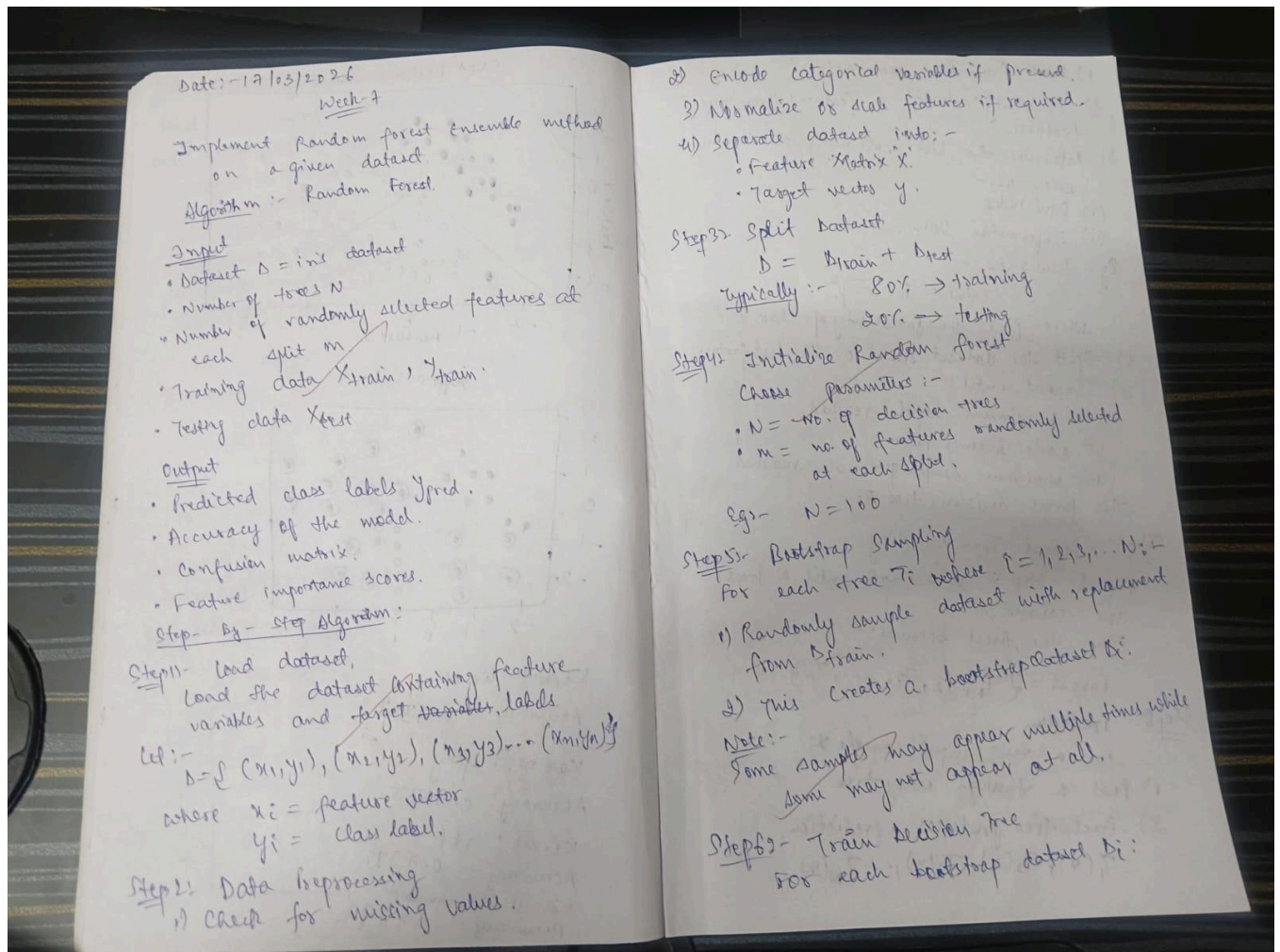
plt.plot(fpr, tpr, label="AUC = %0.2f" %
roc_auc) plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

print("AUC Score:", roc_auc)
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot



- 1) Start from the root node.
- 2) Randomly selected m features from total features.
- 3) Determine the best split using a criterion such as:-
 - (i) Gini Index
 - (ii) Information Gain.

Eg:- Gini Index:-

$$Gini = 1 - \sum p_i^2$$

where p_i is the probability of class i .

- 4) Split the dataset based on the best feature.
 - 5) Repeat until:-
 - (i) Maximum depth reached.
 - (ii) Node becomes pure.
 - (iii) Minimum samples per node reached.
- This forms decision tree T_i .

Step 9:- Repeat Tree Building
Repeat step 5 and step 6 until N trees are created.
Thus the forest becomes:-
Forest = $\{T_1, T_2, T_3, \dots, T_N\}$.

Step 10:- Prediction

- for a new input sample x :
- 1) Pass x through every tree.
 - 2) Each tree produces a prediction.
 $T_1(x), T_2(x), T_3(x), \dots, T_N(x)$.

Step 5:- Majority Voting

For classification problems:-
Final prediction is determined by majority voting
 $Y_{pred} = \text{mode}(T_1(x), T_2(x), \dots, T_N(x))$

For regression:-

$$Y_{pred} = \frac{1}{N} \sum_{i=1}^N T_i(x)$$

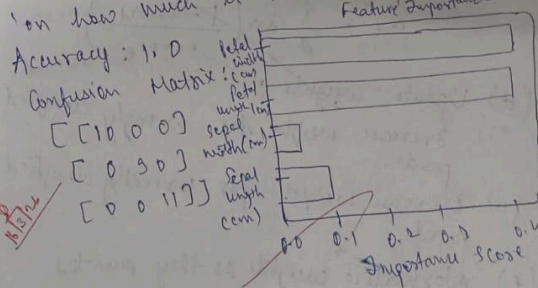
Step 10:- Model Evaluation
Evaluate the model using testing dataset.
Metrics used:-

1) Accuracy:- $\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$

2) Confusion Matrix

	Actual Class 1	Actual Class 2	Actual Class 3
Predicted Class 1			
Predicted Class 2			
Predicted Class 3			

3) Feature Importance:- Random Forest computes importance score for each feature based on how much it reduces impurity.



Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
```

```
df1 = pd.read_csv("iris.csv")
print(df1.info())
df1.head()
```

```
le = LabelEncoder()
```

```

df1['species'] = le.fit_transform(df1['species'])
X = df1.drop('species', axis=1)
y = df1['species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)
y_pred = rf_default.predict(X_test)
default_score = accuracy_score(y_test, y_pred)
print("Accuracy with default n_estimators (10):", default_score)

best_score = 0
best_n = 0

for n in range(1, 101):
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)

    y_pred = rf.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    if score > best_score:
        best_score = score
        best_n = n

print("Best Accuracy:", best_score)
print("Optimal number of trees:", best_n)

```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Week-8 Date: 23/03/2024

Implement Boosting Ensemble Method.

Input:

- Training dataset $D = (x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- No. of models T

Algorithm:

- Initialize weights:
Assign equal weights to all data points
 $w_i = \frac{1}{n}$
- For each iteration $t = 1$ to T :
 - Train a weak learner (eg. decision stump) using weights w_i .
 - Compute error of model:
 $error_t = \sum w_i \cdot I(y_i \neq \hat{y}_i)$
 - Compute model weight:
$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - error_t}{error_t} \right)$$
 - Update weights:
 - Increase weight for wrongly classified points.
 - Decrease weight for correctly classified points.
 - Normalize weights so they sum to 1.
- Final Prediction:

$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right)$

Output

Model Performance

Accuracy: 0.9643

Roc-AUC Score: 0.9525

Confusion Matrix:

$$\begin{bmatrix} 40 & 3 \\ 1 & 70 \end{bmatrix}$$

Classification Report:

	Precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.98	0.93	0.97	71
accuracy			0.96	114
micro avg	0.97	0.96	0.96	114
weighted avg	0.9726	0.96	0.96	114

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix
```

```
df1 = pd.read_csv("income.csv")
print(df1.info())
df1.head()
df1 = pd.read_csv("income.csv")
```

```
X = df1.drop("income_level", axis=1)
y = df1["income_level"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model_10 = AdaBoostClassifier(
```

```

    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=10,
    random_state=42
)

model_10.fit(X_train, y_train)
y_pred_10 = model_10.predict(X_test)

acc_10 = accuracy_score(y_test, y_pred_10)
print("Accuracy with 10 estimators:", acc_10)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_10))

estimators = [10, 20, 50, 100, 150, 200]
accuracies = []

for n in estimators:
    model = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=n,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)
    print(f'n_estimators = {n}, Accuracy = {acc}')
best_index = np.argmax(accuracies)
print("\nBest number of trees:", estimators[best_index])
print("Best accuracy:", accuracies[best_index])

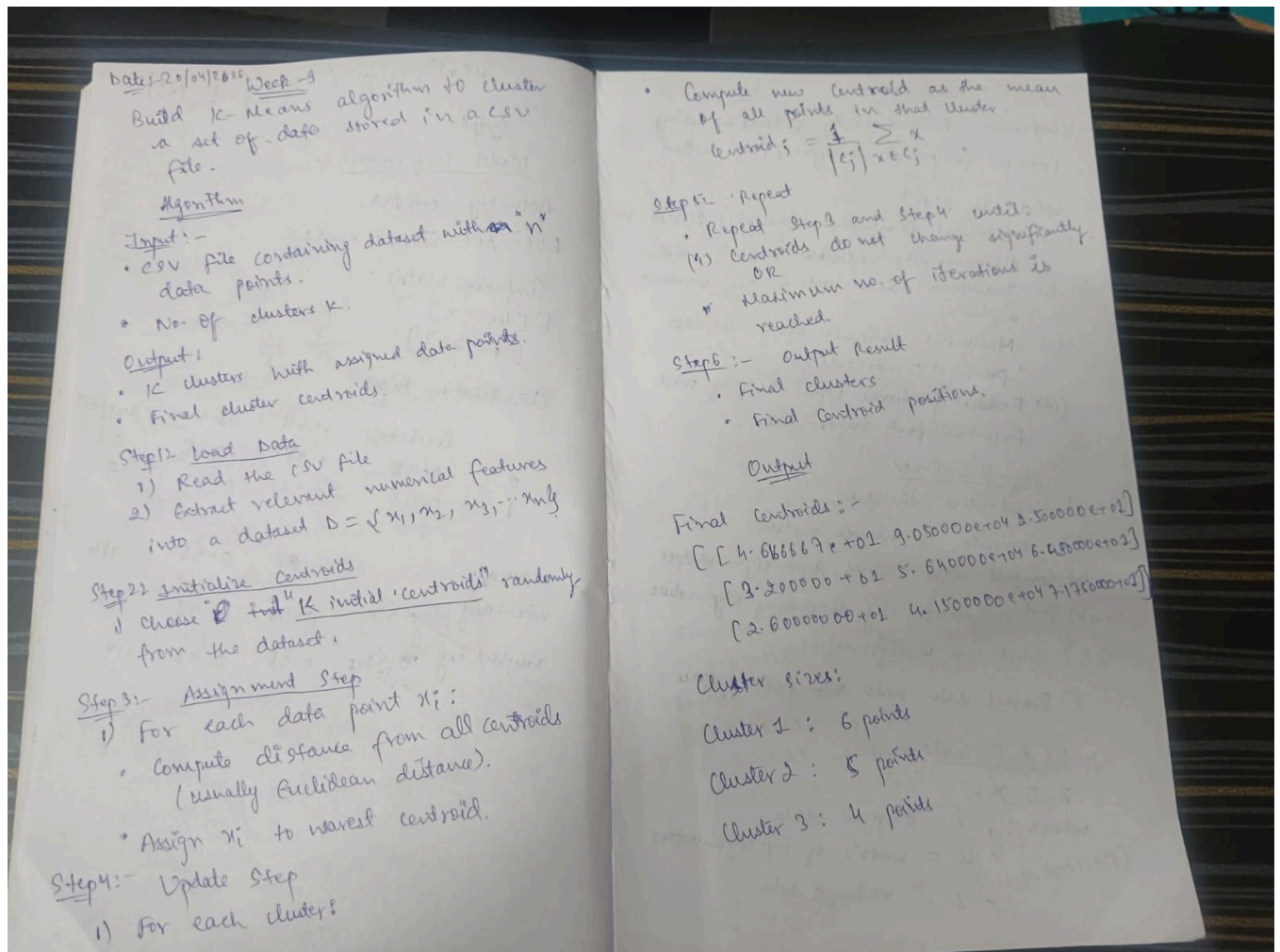
plt.plot(estimators, accuracies, marker='o')
plt.xlabel("Number of Trees")
plt.ylabel("Accuracy")
plt.title("AdaBoost Performance")
plt.grid()
plt.show()

```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

```
df1 = pd.read_csv("iris.csv")
print(df1.info())
df1.head()
```

```
X = df1[['petal_length', 'petal_width']]
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
y_pred = kmeans.fit_predict(X)
```

```
# Add cluster labels to dataframe
df1['cluster'] = y_pred
```

```
plt.figure(figsize=(6,4))
```

```

plt.scatter(X['petal_length'], X['petal_width'], c=y_pred, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            color='red', marker='X', s=200, label='Centroids')
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.title('K-Means Clustering (Petal Features)')
plt.legend()
plt.show()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans_scaled = KMeans(n_clusters=3, random_state=42)
y_pred_scaled = kmeans_scaled.fit_predict(X_scaled)

plt.figure(figsize=(6,4))
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=y_pred_scaled, cmap='viridis')
plt.scatter(kmeans_scaled.cluster_centers_[0], kmeans_scaled.cluster_centers_[1],
            color='red', marker='X', s=200, label='Centroids')
plt.xlabel('Scaled Petal Length')
plt.ylabel('Scaled Petal Width')
plt.title('K-Means with Feature Scaling')
plt.legend()
plt.show()

inertia = []
k_range = range(1, 10)

for k in k_range:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.figure(figsize=(6,4))
plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method')
plt.show()

```

Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot

Week-10

Implement dimensionality reduction using Principal Component Analysis (PCA) method

What is PCA?

Ans: Principal Component Analysis (PCA) is a dimensionality reduction technique that:

- (i) Transforms data into a new coordinate system.
- (ii) Maximizes variance along new axes (principal components).
- (iii) Reduces features while keeping most important feature information.

Algorithm Steps

- 1) Standardize the data.
- 2) Compute Covariance matrix.
- 3) Compute eigen values and eigen vectors.
- 4) Sort eigenvectors by descending eigenvalues.
- 5) Select top k eigenvectors.
- 6) Project data onto new feature space.

PCA Formula

$$Z = XW$$

where :

- X = original data
- W = matrix of top eigenvectors
- Z = reduced data

Output

Original data:

	Age	Income	Spending Score
0	25	40000	60
1	30	50000	65
2	35	60000	70
3	40	80000	30
4	45	90000	25
5	50	100000	20
6	33	38000	75
7	27	41000	80
8	33	58000	68
9	48	85000	32
10	52	110000	18
11	23	40000	72
12	31	52000	66
13	37	62000	55
14	42	78000	35

Mean:

$$[2.6466667e+01 \ 6.606667e+04 \ 5.697222e+01]$$

Standard deviation:

$$[9.0101177e+00 \ 2.2804508e+04 \ 9.2643742e+01]$$

Standardized data:

$$\begin{bmatrix} -1.276234 & -1.875396814 & 0.12087517 \\ -0.114167 & -0.9246802 & 0.64715643 \\ -0.162722 & -0.876375 & 0.87401963 \\ 0.3521574 & 0.6846414 & -0.34055751 \\ \vdots & \vdots & \vdots \\ 0.6141242 & 0.53925308 & -0.71373071 \end{bmatrix}$$

Covariance Matrix:-

$$\begin{bmatrix} 1.07142857 & 1.05469896 & -0.9947825 \\ 1.05469896 & 1.09140857 & -1.0068168 \\ -0.9947825 & -1.0068168 & 1.07142857 \end{bmatrix}$$

Eigen values:

$$[3.10916874 \quad 0.01606649 \quad 0.08905046]$$

Eigen vectors:

$$\begin{bmatrix} -0.57965039 & -0.66592574 & 0.46962574 \\ -0.58189034 & 0.7417262 & 0.3235703 \\ 0.57046696 & 0.0790913 & 0.81742432 \end{bmatrix}$$

Sorted Eigen values

Selected Top K Eigen vectors (W):-

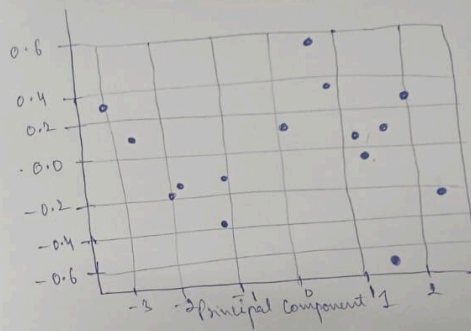
$$\begin{bmatrix} -0.57965039 & 0.46962574 \\ -0.58189034 & 0.3235703 \\ 0.57046696 & 0.81742432 \end{bmatrix}$$

Reduced Data (After PCA):-

$$\begin{bmatrix} 1.6616261 & -0.64623275 \\ 1.20690058 & -0.049375621 \\ 0.75217506 & 0.54672032 \\ -1.12354916 & -0.37502849 \\ -1.84866259 & -0.149529029 \\ \vdots & \vdots \\ -1.0763313 & -0.115466697 \end{bmatrix}$$

Explained Variance Ratio:

$$[0.96729695 \quad 0.02970459 \quad 0.00499846]$$



Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA

dfl = pd.read_csv("heart.csv")
```

```

print(df1.info())
df1.head()
df = df1.copy()

le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex']) # M=1, F=0
df['ExerciseAngina'] = le.fit_transform(df['ExerciseAngina']) # Y=1, N=0

df = pd.get_dummies(df, columns=['ChestPainType', 'RestingECG', 'ST_Slope'])

df.head()

X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scaling
scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train) X_test_scaled =
scaler.transform(X_test) models = {
    "SVM": SVC(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier()
}

results = {}

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    results[name] = acc
    print(f"{name} Accuracy: {acc:.4f}")

best_model_name = max(results, key=results.get)
print("\nBest Model:", best_model_name)

# Apply PCA (keep 95% variance OR choose components like n_components=5)
pca = PCA(n_components=0.95)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

```

```
print("Original shape:", X_train_scaled.shape)
print("Reduced shape:", X_train_pca.shape)

# Train again on PCA data
pca_results = {}

for name, model in models.items():
    model.fit(X_train_pca, y_train)
    y_pred = model.predict(X_test_pca)
    acc = accuracy_score(y_test, y_pred)
    pca_results[name] = acc
    print(f'{name} (PCA) Accuracy: {acc:.4f}')

best_pca_model = max(pca_results, key=pca_results.get)
print("\nBest Model after PCA:", best_pca_model)

print("\n--- Comparison ---")
for model in models.keys():
    print(f'{model}: Before PCA={results[model]:.4f}, After PCA={pca_results[model]:.4f}')
```